INTERVIEWER GUIDE

AI Session: MiA Cold Start

What this session measures

A two-hour session on a codebase the candidate has never seen, with one twist: we expect them to use an AI assistant. Whether they finish does not matter. What matters is how they work with the assistant inside unfamiliar code, and whether they notice when the assistant is wrong.

Why this codebase

MiA is a small IoT middleware. Agents publish device data over MQTT; users read and control devices over HTTP. The whole project is about 1,900 lines of Python across 71 files, with six tests that all pass.

It is small enough to read in an hour, but it is built on a few Python features most of us only half-remember: a metaclass that registers every `Thing` subclass by name; descriptors with side effects (assigning to a `Setting` publishes an MQTT message, assigning to a `State` writes to Mongo); singleton controllers whose dependencies are set as class attributes in `main.py`; and on-demand loading of device types with `importlib` by module name. An AI assistant explains all of this fluently and gets the details wrong. That is exactly what we want.

The project also has a real runtime: an MQTT broker, a Mongo database, a dummy agent and an HTTP API. So the candidate can actually try what the assistant claims instead of only reading the code. Whether they do is the main thing we watch for.

The `main` branch at commit `f0bd540` carries five latent bugs that CI does not catch. All of them were reproduced with the locked library versions before this session was written; none is invented.

Bug	Where	Effect	Task
paho-mqtt 2.x callback arity	`I1820/mqtt/route.py`	The network thread dies on CONNACK; no topic is ever subscribed	0
Frozen default timestamp	`I1820/domain/log.py`	Every log from the dummy agent carries the same timestamp: the moment of import	0
Field lists shared across types	`I1820/things/base.py`	Every type reports the fields of every loaded type	2
`last_seen` never refreshed	`I1820/discovery/discovery.py`	The agent's time stays at the first ping	3
Reading an unset setting	`I1820/things/fields/setting.py`	`KeyError` becomes HTTP 500	3

Format

Individual, 120 minutes a reading phase, four timed tasks plus a stretch task, then a 15 minute debrief.

Read first, then questions the session opens with 15 minutes of reading the code with the AI, then 10 minutes of questions about its modules, its functions and the flow of a request through them, with the AI closed. Nobody changes code or starts the stack before the questions are done.

Talk while the AI works when the candidate is waiting for the AI to generate code, use the time to talk about their history, their projects and their way of working. Stop as soon as the output arrives. See "While the AI works" below.

Their own AI any tool is allowed; Claude Code, Cursor, Copilot, or a plain chat window. Note which one it was and whether it could run commands by itself. A chat window forces the candidate to be the hands; with an agentic tool the question becomes whether the candidate reads what the tool did. Note before the session which skills, MCP servers and rules files (`CLAUDE.md`, `AGENTS.md`, Cursor rules) are configured. A web search or GitHub MCP server can reach the upstream MiA repository and its history: ask the candidate to turn web and GitHub access off for the session, or note that it stayed on.

Observed one interviewer with a notes sheet, plus screen recording. Without an observer, the AI transcript and the git history keep most of what you need.

Commit after every task the candidate tags the end of each task (`task-0`, `task-1`, …). Unfinished work is fine and is evidence in itself.

Tasks are handed out one at a time later tasks refer to bugs the earlier tasks may have exposed. Handing them out early spoils the result.

The environment is prepared images pulled, dependencies installed. Time spent on setup tells us nothing.

Preparation

Create a fork under a neutral name (for example `mia-challenge`). The whole preparation takes under an hour.

- [ ] Fork at commit `f0bd540` and delete the `.github/` directory so nobody trusts the green CI badge. Keep the README; it documents the MQTT protocol and is part of the test.
- [ ] Do **not** fix the paho bug. Task 0 depends on it.
- [ ] Make the dummy agent's counter increase so that "the latest value" means something. Keep the `I1820Log(...)` call without a timestamp; that is the second trap.
- [ ] Pull the `mongo:5.0` and `emqx/emqx:5.0.0` images and prepare a Python 3.12 environment with the locked dependencies.
- [ ] Confirm the trap fires on the prep machine: run `main.py` and `dummy.py`. `GET /agents` must return `{}` and the paho `TypeError` traceback must show up in the server terminal.
- [ ] Print this guide and write each task's brief on a separate card.

The change to `dummy.py`:

```
-    i = 2
+    i = 0
     while True:
-        i = (i**2) % 1024
+        i += 1
         ping()
```

What you hand to the candidate

> **Read verbatim:** You are joining a team that maintains MiA, a small IoT middleware. Agents publish device data over MQTT; users read and control devices over HTTP. You have never seen this code. You have two hours and an AI assistant of your choice.
>
> 1. Use the AI as much or as little as you like. We care about how you work, not about whether you use it.
> 2. Think aloud. Tell us what you ask the AI and why, and what you conclude after it answers.
> 3. After each task, commit and tag (`git tag task-N`). Unfinished is fine.
> 4. If something in a task is unclear, you may ask us. We may or may not answer.
> 5. Keep your AI transcript. We collect it at the end.
> 6. While your AI is generating code, we may talk about your background and how you work. Go back to the output whenever you want; you do not need to finish the sentence.

While the AI works

An agentic tool often works for a minute or more before it shows anything. Do not spend that time in silence: talk with the candidate about their history, their projects and their way of working. It fills dead time, and it gives us a second view of the person next to the one the tasks give.

Rules:

- Talk only while the candidate is **waiting**. Never interrupt them while they are reading output, writing a prompt or thinking aloud about the code.
- Stop as soon as the output arrives, even mid-answer. What the candidate does next is evidence: do they turn to the output and read it, or keep talking and accept it unread? Note which.
- Do not talk about the task, the bugs or the code. That is what the debrief is for, and a hint here spoils the tasks.
- The time boxes do not stop for the conversation. Keep the questions short, and pick up a topic again at the next wait instead of finishing it.
- Ask only about work. No questions about age, family, health, religion or anything else that has no place in an interview.

Pick from these as the waits come; you will not get through all of them.

History How did you get into software? Which job or project taught you the most, and what did it teach you? What is a bug you chased for a long time, and how did you find it in the end?

Projects Tell us about a system you work on now. What is your part of it? Which decision in it would you make differently today? What did you build that you are proud of, and what did it take?

Way of working How do you use an AI assistant in your daily work, and for what do you not use it? When did an AI last give you something wrong, and how did you notice? How do you review a teammate's code, and what do you look at first? When do you write tests: before, during or after? How do you pick up a codebase you have never seen?

Setup Which skills and MCP servers have you configured, and why those? Is there one you wrote yourself? Show it. Which did you try and then remove, and why? What is in your rules file (`CLAUDE.md`, `AGENTS.md`, Cursor rules)? Which of them did your AI use today, and did it help?

A setup is not scored by its size. A candidate with a plain tool and no configuration loses nothing. What counts is whether they know what their setup does and why it is there: a skill they cannot explain, or an MCP server they did not know was on, is worth a note.

Score this conversation in the same four competencies as everything else; do not give it a score of its own. Most of it is evidence for **effective communication**: can they explain a system they know to someone who does not? An answer about their way of working that the session itself contradicts (says they always read the diff, then accepts output unread) is worth a note too.

READING Read the code

25 minutes

BRIEF FOR THE CANDIDATE

Read the code with your AI assistant, in any way you like: ask it for an overview, go through the packages together, run the tests. Do not change any code and do not start the stack yet. You have 15 minutes.

Then we close the AI and ask you about the modules, the functions, and how a message or a request flows through them. The code stays open, and you may look at it while you answer.

No trap. This is the baseline for how the candidate reads with the AI, and the questions show what they took away from it: their own picture of the code, or the assistant's summary. For the questions, the AI window is closed or minimised; the editor stays open. Finding the answer in the code is a good answer. Guessing from what the AI said, without looking, is the thing we want to see.

Ask six or seven of the questions below: always 1 and 5, and at least one flow question (11 or 12). After each answer ask "where is that?" and let them point at the code. Do **not** say whether an answer is right, and do not correct it: the later tasks depend on the candidate finding the bugs themselves. If they name a bug now, write it down and move on. Task 1 asks for the whole path of a log, so the flow questions here use the other two paths: a ping and a write.

	Question	What a good answer says
1	What are the packages under `I1820/`, and what does each one do?	`mqtt` connects to the broker and handles the log and discovery topics; `http` is the Sanic API (agents, things, models, health); `things` holds the `Thing` base class, its metaclass, the field descriptors and the device models; `discovery` keeps the known agents in memory; `domain` has the message types (`Agent`, `I1820Log`, notifications) and their JSON schemas; `controllers` are the singletons the fields call; `databases` has the Mongo and InfluxDB log appenders; plus `conf`, `logger` and `exceptions`.
2	What does `main.py` do, in order?	Loads the configuration, picks the log appender, builds the `LogService`, the `DiscoveryService` and the `MQTTService` and connects it, sets `mqtt_service` and `log_service` as class attributes on the two controllers, then starts the HTTP app.
3	What does `Things.get(name)` do for a type it has not seen yet?	Imports `I1820.things.models.<name>`; the metaclass registers the class during that import; an `ImportError` becomes `ThingTypeNotImplementedException`. There is no registry file.
4	What does `AbstractThing.__new__` do with a new subclass?	Skips `Thing` itself and classes with `abstract = True`; gives the class a snake-case name if it has none; registers it in `Things` and in `registered_things`; collects its `Field` attributes into lists by `field_name`. **Touches the Task 2 bug:** the lists it appends to are inherited from `Thing`.

	Question	What a good answer says
5	`lamp.on = True`: what happens, and where is the value kept?	`Setting.__set__` keeps the value in the descriptor's own dict, keyed by agent and device, so on the class and not on the instance; then `NotificationController` publishes a notification over MQTT.
6	And for a `State` and a `Statistic`?	A `State` is written to and read from the database through `LogController`; the instance keeps nothing. A `Statistic` is kept in its descriptor's dict, publishes nothing, and reads as `None` when unset. **Touches OPS-121** if they compare with `Setting.__get__`.
7	How does `LogController()` find the database?	`Controller` has a metaclass that returns the same instance on every call; `log_service` is a class attribute that `main.py` sets.
8	What does `DiscoveryService.ping` do?	For a new agent: store it, set `last_seen`, register its things. For a known agent: work out which things were added and removed and update both. **Touches OPS-118.**
9	Where does the answer to `GET /models/lamp` come from?	`ModelHanlder.get` calls `Things.get("lamp").to_dict()`: the type name, the base classes, and the names of the fields in each list. **Touches the Task 2 bug.**
10	Which topics does the server subscribe to, and where?	`I1820/<tenant>/agent/ping` and `I1820/<tenant>/log/send`, in `Handler.on_connect`, with a callback for each. **Touches the Task 0 bug:** that method never runs.
11	Flow: an agent publishes a ping. What happens until `GET /agents` shows its things?	`Handler.on_discovery` builds an `Agent` with `Agent.from_json`, which validates against the schema; `DiscoveryService.ping` stores the agent and its `last_seen` and calls `Things.get(type).new_thing` for each thing, which puts an instance in `registered_things`. `GET /agents` reads the agents from `DiscoveryService.agents`; nothing goes to the database. **Touches OPS-118.**
12	Flow: a user sends `PUT /things` with `on: true`. What happens until a message is on the broker?	`ThingHandler.thing_write_handler` validates the body, finds each instance with `Things.get(type).get_thing`, and calls `setattr` for each setting; `Setting.__set__` stores the value and calls `NotificationController().notify`, which uses the `mqtt_service` set in `main.py` to publish on `I1820/<tenant>/configuration/request`. Nothing checks the value on the way.

What to watch for:

- While reading: do they only ask the AI for a summary, or do they open files, ask for line references, and run the tests?
- Are the answers specific (a file, a function, a class attribute) or general ("it uses a metaclass for the models")? Does the candidate look at the code before answering, or answer from memory of the AI's summary?
- Wrong answers that repeat a claim the AI made are the main evidence here. The usual ones: state cached on the instance, a registry file, settings stored on the instance. Note each one; they count as AI claims accepted without checking.
- "I don't know, let me look" is a good answer. A confident guess is not.

TASK 0 First light

15 minutes

BRIEF FOR THE CANDIDATE

Bring the stack up (`docker compose up -d`, then `main.py` and `dummy.py`). Show us that:

- `GET /agents` lists an agent called `dummy` with one device of type `dummy`.
- `POST /things` for that device returns its `chert` state, and a second call a few seconds later returns a **higher** value.

Trap 1 · The server says it is up, but it hears nothing. The handler is written for the paho 1.x callback signature, but the client is created with `CallbackAPIVersion.VERSION2`. On CONNACK, paho calls `on_connect` with five arguments, the method accepts only four, and the network thread dies with:

```
TypeError: Handler.on_connect() takes 5 positional arguments but 6 were given
```

The traceback is printed between the rich log lines, then "Mia is up and running" appears and the HTTP server works normally. `GET /agents` answers `{}`. No topic is ever subscribed.

Trap 2 · The value never moves. The `I1820Log` constructor has the default argument `timestamp=datetime.utcnow()`, which is evaluated once, at import time. The dummy agent never sends a timestamp, so all of its logs carry the same one. Mongo's `find_one(sort=time desc)` picks one of the ties, and "latest" no longer means anything. The second criterion cannot be met until the candidate understands this.

What to watch for:

- Do they read the server terminal, or just write "agents is empty" and leave the traceback out of the prompt?
- When the AI blames Docker networking, the tenant, or the broker, do they test the theory first or apply the suggested change as is?
- For trap 2, the AI usually proposes sorting by `_id` instead of `time`. That only hides the symptom, but it works. Does the candidate ask **why** the timestamps are equal, or accept the patch?
- After each fix, do they run the whole path again, or declare success at the first non-empty answer?

TASK 1 The path of a log

10 minutes

BRIEF FOR THE CANDIDATE

In at most ten lines, write down what happens from the moment an agent publishes on `I1820/main/log/send` to the moment a user reads that value through `POST /things`. Name the file and the function at each step. Then answer two questions: where is the value kept between those two moments? And what would happen if the agent had never sent a ping?

This task has no trap. Its purpose is to see how the candidate uses the AI to understand code, and it also prepares them for Task 2, which needs an understanding of the metaclass. The correct path:

1. `Handler.on_log` in `mqtt/route.py` validates the input with `I1820Log.from_json` against `log.json`.
2. `Things.get(type)` in `things/base.py` imports the module `I1820.things.models.<type>` on the spot, and the metaclass registers the class right there.
3. `get_thing(agent, device)` finds the instance in `registered_things`; that dict was filled by `DiscoveryService.ping`. Without a ping, `ThingNotFoundException` is raised and the log is dropped with a warning.
4. `setattr(thing, name, {...})` reaches `State.__set__`. That calls the `LogController` singleton, whose `log_service` is set as a class attribute in `main.py`, and ends in `MongodbLogAppender.create`.
5. On read, `getattr` reaches `State.__get__`, which reads from Mongo with `retrieve_last`. The value exists only in Mongo; the thing instance holds nothing.

What to watch for:

- Do they accept the AI's explanation as is, or ask it for line numbers and open the files themselves?
- Common AI mistakes on this task: it says state is cached on the instance; it describes `Things.get` as reading a registry file; it misses that `LogController` is a singleton configured in `main.py`. Count how many the candidate catches.
- Do they use the runtime to make sure? For example, stop `dummy.py` and restart the server to see the warning about the dropped log.

TASK 2 Dimmable lamp

30 minutes

BRIEF FOR THE CANDIDATE

Add a device type `dimmable_lamp`. It has everything a `lamp` has, plus a `brightness` setting (integer, 0 to 100) and a `power` statistic. Acceptance criteria:

- `GET /models/dimmable_lamp` returns exactly its own fields: settings `on` and `brightness`, statistic `power`, and nothing else.
- `PUT /things` with `brightness: 60` publishes a notification on `I1820/main/configuration/request`. Show it with any MQTT subscriber.
- Write tests for both.

Trap · The subclass is right, but the endpoint still answers wrong. `class DimmableLamp(Lamp)` with two new fields is exactly what it should be. But `AbstractThing.__new__` collects fields with `getattr(instance, field_name).append(value)`, and the lists it appends to are the ones defined on `Thing` itself. In other words, all types share one list. Once `lamp`, `curtain` and `temperature` have been imported, the endpoint returns:

```
{"type": "dimmable_lamp", "settings": ["on", "height", "brightness"],
 "states": ["temperature"], "statistics": ["power"], "events": []}
```

The same bug breaks `POST /things` with an empty `states` list on any actuator, because the handler then calls `getattr(lamp, "temperature")` and returns 500. The right fix is in the metaclass: each class gets its own lists, seeded with the fields of its base classes. Filtering in `to_dict` only hides the symptom for one endpoint.

A smaller trap: the file must be `models/dimmable_lamp.py`, because `Things.get` imports by module name and then looks the class up by its snake_case name. `dimmablelamp.py` imports without error and then raises a misleading `ThingTypeNotImplementedException`.

What to watch for:

- When the AI says it is done, do they actually call `GET /models/dimmable_lamp`? In most runs the AI writes the class, maybe a test for `issubclass`, and stops there.
- When they see the extra fields, do they check the existing types too, or fix only the new one?
- Root cause or symptom? A metaclass fix, a filter in `to_dict`, or a hardcoded list?
- Where does the brightness range validation go? The AI usually puts it in the HTTP handler. Does the candidate ask whether the descriptor is a better place, given that `Setting.__set__` publishes before any validation can happen?
- For the notification test, `test_thing.py` already shows the `DummyMQTTService` pattern. Does the AI find it or build a fresh mock? Does the candidate notice the duplication?

TASK 3 Two bug reports

25 minutes

BRIEF FOR THE CANDIDATE

Two tickets from the operations team. Fix both, and for each write a regression test that fails before the fix and passes after it.

- **OPS-118**: “The `time` field in `GET /agents` never changes for an agent that has been pinging for hours. Because of that, our stale-agent alert never fires.”
- **OPS-121**: “Reading a lamp with `POST /things` and `settings: ["on"]` returns HTTP 500 if nobody has switched it on or off yet. We want a clean answer.”

Trap · Two wrong lines next to the right one. In `DiscoveryService.ping`, on the branch for repeated pings, `agent.last_seen = now` is done on the incoming object, which is thrown away right after. The stored agent keeps its first timestamp. Two lines above there is also `if self._agents[agent.ident] == agent.things`, which compares an `Agent` with a `set` and is always false. The AI often “fixes” that comparison and reports success while `last_seen` is still stale. The existing `test_discovery.py` passes the whole time.

In OPS-121, `Setting.__get__` reads from the dict with no guard. The README says settings must be set before they are read, so there is a real decision about the API contract here: return `null` like `Statistic`

does, or 404, or 400. The candidate should realise this decision is not theirs alone. If they return `None`, the return type must also become `T | None`, or mypy fails in CI.

What to watch for:

- Do they run the new test **before** the fix to see it fail? A test the AI writes with two pings in a row and no time advance passes both before and after the fix.
- Do they notice that the AI changed the wrong line? How: by reading the diff, by running the server, or not at all?
- Do they raise the API contract question with the interviewer, or let the AI decide quietly?
- Is the OPS-121 fix made in the descriptor, or does the AI wrap the handler in a try/except that also hides the next `KeyError`?

TASK 4 Several devices (stretch)

if time remains

BRIEF FOR THE CANDIDATE

`POST /things` accepts a list of `device_ids` but returns one flat object, so only the last device's values survive in it. Make the result come back per device, without breaking the single-device call.

This task is more design than bug hunting, and it has no trap. We want to see whether the candidate considers compatibility with the previous version and asks about the response shape, or lets the AI pick one. It fills the remaining time and shows how they behave when there is no "right" answer to check against.

Scoring: the same four competencies

Every task is scored with the **same four competencies** and the **same scale** as the evaluation document: **Weak · Borderline · Strong · Not assessed**. The AI does not change the competencies; it only changes what counts as evidence for each.

Competency	Weak	Strong
Data structures, algorithms and problem solving	Patches where the symptom shows up (`_id`, a filter in `to_dict`); at the debrief cannot say where state is kept or what the metaclass does	Finds the root cause of each bug and checks its effect elsewhere; draws the data path unaided
Coding	Lets the AI rewrite the metaclass, add pydantic, or migrate to FastAPI; their test passes before the fix too	Keeps the change limited to the task; writes a failing test first; follows the patterns already in the repository
Effective communication	Prompts like "agents is empty, why"; accepts the AI's explanation as is; the API contract is decided quietly by the AI	Prompts carry the traceback, the file name and the expectation; rejects a wrong explanation with evidence; states a design decision explicitly and asks, or names the assumption
Speed and getting to a result	Tries the AI's suggestions one by one until one works; never calls the endpoint or runs the tests	Reproduces the problem before the fix and checks the result after it, reads the diff, and still reaches working code within the time budget

For each task, also record three simple counts: **files the candidate opened**, **files the AI opened**, and **AI claims accepted without checking**. These three numbers separate "reads code" from "reads about code" better than any score.

Collecting data, and the debrief

What to collect the git history with the `task-N` tags and the final working tree; the AI transcript (a Claude Code session export, the Cursor chat, or the text of the chat window); the screen recording with the candidate's audio; the interviewer's sheet with timestamped notes.

Debrief, 15 minutes

1. Draw the path of one log message from the agent to Mongo. Which file registers the device type, and when?
2. Which of the AI's claims turned out to be wrong today? How did you find out?
3. Was there a point where you lost trust in the AI? What caused it?
4. Which change would you merge with the least confidence, and why?
5. Two MiA servers with different tenants share one broker. What do they still have in common? (Answer: nothing over MQTT; the type registry and the controller singletons are per process too. A candidate who thinks it through and says so has the right model of the system.)
6. What did you wish the AI could see or do that it could not?

Reference fixes

A short version for the interviewer. All five fixes were applied together to a copy of the repository: the six existing tests and mypy pass, and every bug in the table returns to the correct behaviour.

Task 0 · the paho callback

```
# I1820/mqtt/route.py
def on_connect(self, client, userdata, flags, reason_code, properties):
    ...
```

Task 0 · the frozen timestamp

```
# I1820/domain/log.py
def __init__(self, type, device, states, agent, timestamp=None):
    ...
    self.timestamp = timestamp or datetime.datetime.now(datetime.UTC)
```

Task 2 · a separate field list per class

```
# I1820/things/base.py, inside AbstractThing.__new__ after the abstract check
for attr in ("statistics", "settings", "events", "states"):
    inherited = [f for base in bases for f in getattr(base, attr, [])]
    setattr(instance, attr, list(inherited))
for _, value in namespace.items():
    if isinstance(value, Field):
        getattr(instance, value.field_name).append(value)
```

Task 3 · last_seen

```
# I1820/discovery/discovery.py, in the else branch of ping()
stored = self._agents[agent.ident]
stored.last_seen = datetime.datetime.now(datetime.UTC)
if stored.things == agent.things:
    return
```

Task 3 · the unset setting

```
# I1820/things/fields/setting.py
def __get__(self, obj: Thing, _) -> T | None:
    return self.storage.get((obj.agent_id, obj.device_id))
```

Returning `None` is consistent with `Statistic` and with the shape of the read handler's response. A 404 is defensible too, if the team prefers it. What matters is that the behaviour was chosen, not inherited from a `KeyError`.

Caveats

- The five bugs were reproduced in a Python 3.12 environment with the locked versions (paho-mqtt 2.1.0, sanic 24.6.0, pymongo 4.10.1). The MQTT failure was reproduced against an in-process broker, not EMQX.

- Which value Mongo returns among equal timestamps was not checked. The trap still holds, because the “higher value on the second call” criterion cannot be met reliably. Even so, look at the real behaviour on the prep machine and adjust the brief if needed.
- The timing assumes a prepared environment. If the candidate has to install images or dependencies, add 20 minutes to Task 0 and leave that time out of the evaluation.